

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

B.Tech (Cybersecurity)

DATABASE SYSTEMS LAB

BCSE302L

PROJECT REVIEW 1

Problem Analysis and Database Design

Project Title

Smart Hostel Management System

Team Members	
Harjot Singh Bagga	24BYB0115
Tanuj Sharma	24BYB0122
Daksh Khanna	24BYB0125

Index

S. No.	Component	Page Number
1	Problem Definition and Scope	3 – 4
2	Requirement Analysis	5 – 6
3	ER Diagram	7 – 8
4	Relational Schema	9 – 12

1. Problem Definition and Scope

A. Problem Definition

In most educational institutions, hostel operations are still handled using paper registers, notice boards and separate spreadsheets. Room allotment, maintenance complaints, leave applications and visitor entries are each recorded in a different place, and these records are rarely linked to one another. As a result, students face delays in getting rooms allocated, complaints resolved, leave approved and visitor permissions granted, while wardens and maintenance staff find it difficult to keep track of daily activities.

From a data point of view, the main problems are:

- There is no single, reliable record of which student occupies which bed in which room, so vacancy information is often outdated and the same bed can be promised to two students.
- Complaints do not have a clear owner or status history, so it is not known who is handling a complaint or how long it has been pending.
- Leave and visitor approvals are recorded on paper and are not linked to the student record or to the warden who approved them.
- Because the data is scattered, producing occupancy, complaint or leave reports for the hostel administration requires manual effort and is prone to errors.

B. Proposed Solution

The Smart Hostel Management System (SHMS) is a web platform that brings the major hostel operations together in one place. This project designs the **relational database** at the core of SHMS. All hostel data is stored in eight related tables: STUDENT, HOSTEL, ROOM, STAFF, ROOM_ALLOCATION, COMPLAINT, LEAVE_REQUEST and VISITOR_PASS. Every request raised by a student is linked to that student through a foreign key, and every action taken by an administrator, warden or maintenance worker is linked to the corresponding STAFF record. Each request carries a status attribute, so its progress can be tracked from submission to closure. Primary key, foreign key, UNIQUE, CHECK, NOT NULL and DEFAULT constraints keep the data consistent, and the same design will be used for the SQL queries, procedures, functions, cursors and triggers in later reviews.

C. Objectives

- Maintain a central database of students, hostels, rooms and staff.
- Record room allocations bed-wise and keep room vacancy accurate at all times.
- Prevent over-allocation of rooms and multiple active allocations for the same student.
- Allow students to raise complaints and track them until they are resolved by maintenance staff.
- Digitize leave applications and visitor passes along with their approval records.
- Keep a complete history of allocations and requests for reports and analysis.
- Support role-based access by storing the role of every staff member.

D. Users / Actors

1. Student

- Registers and maintains personal and academic details.
- Views the allotted hostel, room and bed.
- Raises maintenance complaints and tracks their status.
- Applies for leave and requests visitor passes, and checks their approval status.

2. Admin / Warden

- Manages student, staff, hostel and room records.
- Allocates rooms and beds to students and monitors vacancy.
- Assigns complaints to maintenance staff.
- Approves or rejects leave applications and visitor passes.
- Views occupancy, complaint and leave reports.

3. Maintenance Staff

- Views the complaints assigned to them.
- Updates complaint status (Assigned, In Progress, Resolved).
- Records the resolution date and remarks on the work done.

E. Scope

The database covers the following workflow:

Student Registration → Hostel and Room Setup → Room Allocation (bed-wise) → Complaint / Leave / Visitor Requests → Assignment and Approval by Staff → Status Updates → Hostel Records and Reports

Included in the database design: student, hostel, room and staff records; bed-wise room allocation with allocation history and vacancy tracking; complaint registration, assignment and resolution; leave applications and approvals; visitor passes with QR-code based entry and entry/exit times; request status tracking that drives notifications; and the staff role used for role-based access.

Outside the core schema for this review: fee tracking, announcements, inventory, laundry, lost-and-found, events and the hostel marketplace are separate application modules of SHMS. They are not part of the eight-table core design and can be added as extension tables in later work without changing the existing relations.

2. Requirement Analysis

2.1 Major Functional Requirements

Requirement	Description
Student Management	Store student details: registration number (unique), name, email, phone, gender, year, branch and CGPA.
Hostel Management	Store each hostel block with its name, gender category, location and number of floors.
Room Management	Store rooms of each hostel with room number, floor, room type, capacity and current occupancy.
Room Vacancy Tracking	Show available beds (Capacity − Occupied_Count) for each room, filtered by hostel or room type.
Room Allocation	Allow the admin/warden to allot a specific bed in a room to a student, recording the allocation date and the staff member who made it.
Allocation Control	Prevent allocation beyond room capacity, into a hostel of the wrong gender category, or more than one active allocation per student.
Room Vacating and History	Mark allocations as Vacated or Cancelled with a vacate date while keeping past allocations for records.
Staff Management	Store admins, wardens and maintenance staff with their role, specialization, contact details and hostel.
Complaint Management	Allow students to raise complaints with category, description and priority; record the creation time.
Complaint Assignment and Tracking	Allow the warden to assign a complaint to maintenance staff, and staff to update its status, resolution date and remarks.
Leave Management	Allow students to apply for leave with start date, end date, reason and destination; record approval or rejection and the approving warden.
Visitor Management	Allow students to request visitor passes with visitor details, purpose and visit date; record approval, a unique QR code and entry/exit times.
Status Updates and Notifications	Maintain a status for every allocation, complaint, leave and visitor record; status changes are used to notify the student.
Role-Based Access	Store the role of each staff member (Admin, Warden, Maintenance) so that each user can access only the relevant records.
Reports	Generate hostel occupancy, complaints by category and status, staff workload, and leave and visitor summaries.

2.2 Non-Functional Requirements

Category	Description
Security	Student and staff data must be accessible only to authorized roles. Passwords are stored as hashes at the application layer and database access is restricted by role.
Data Integrity	Every relationship is enforced by a foreign key; NOT NULL, UNIQUE, CHECK and DEFAULT constraints prevent invalid or duplicate data.
Reliability	A bed must never be allocated to two students. Allocation and occupancy updates are performed in a single transaction so that partial updates cannot occur.
Performance	Frequently searched columns (foreign keys, Status, Registration_No) are indexed so that vacancy and request lookups return quickly.
Scalability	The design supports multiple hostels, rooms and staff, and a growing history of requests, without structural change.
Usability	Coded values such as status and category use a small fixed set of readable values so that the interface and reports stay simple.

2.3 Identified Entities and Key Attributes

The functional requirements above lead to the following eight entities, which form the basis of the ER diagram.

Entity	Purpose	Key Attributes
STUDENT	Hostel residents	Student_ID, Registration_No, Name, Gender, Year, Branch, CGPA
HOSTEL	Hostel blocks	Hostel_ID, Hostel_Name, Gender_Type, Location
ROOM	Rooms within a hostel	Room_ID, Room_Number, Room_Type, Capacity, Occupied_Count
STAFF	Admins, wardens and maintenance staff	Staff_ID, Name, Role, Specialization
ROOM_ALLOCATION	Bed allotted to a student in a room	Allocation_ID, Bed_Number, Allocation_Date, Status
COMPLAINT	Maintenance complaints	Complaint_ID, Category, Priority, Status
LEAVE_REQUEST	Leave applications	Leave_ID, Start_Date, End_Date, Status
VISITOR_PASS	Visitor entry permissions	Pass_ID, Visitor_Name, Visit_Date, QR_Code, Status

3. ER Diagram

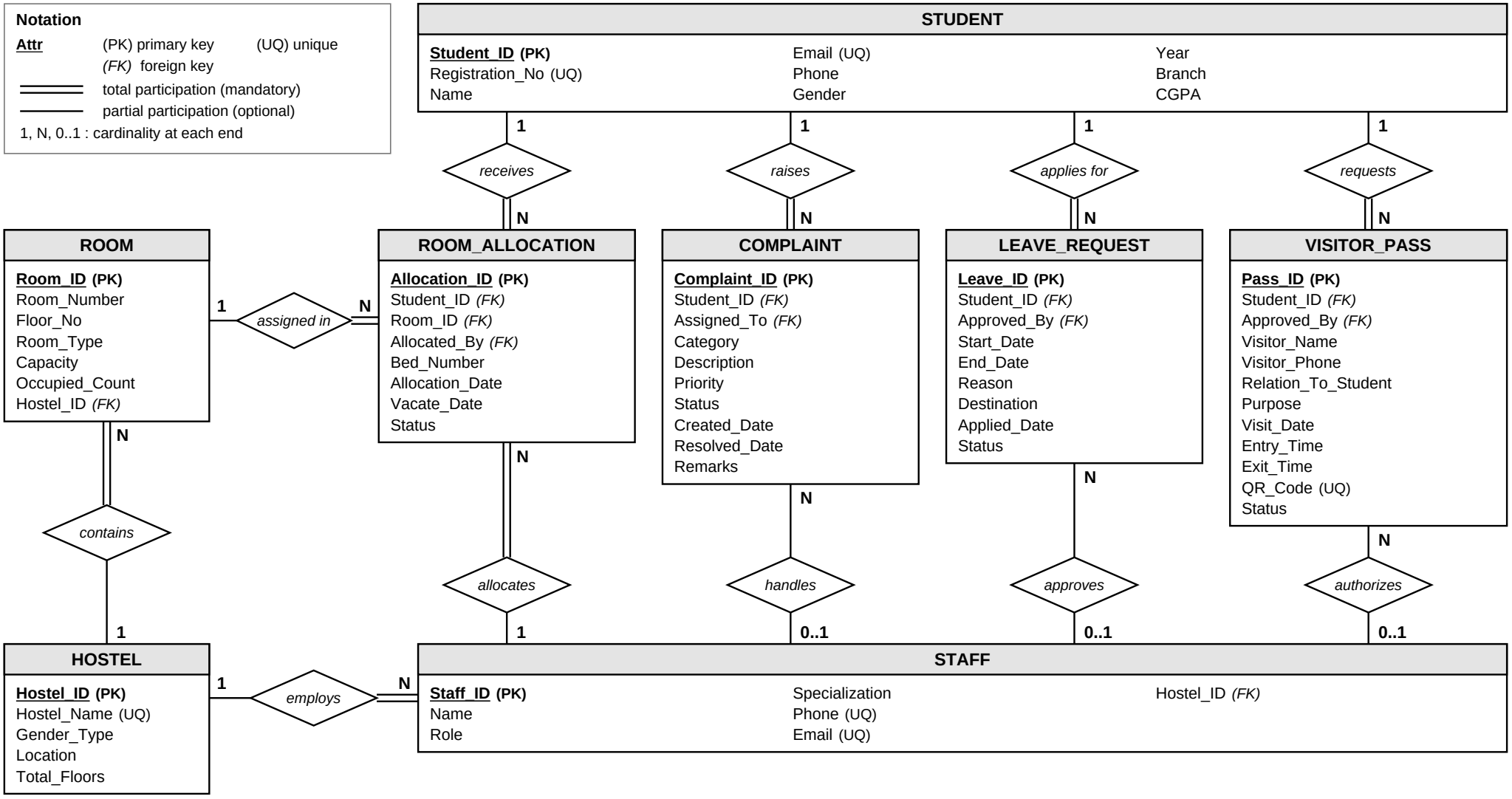


Figure 1: Entity-Relationship Diagram of Smart Hostel Management System

3.1 Relationships and Cardinality

The table below lists every relationship shown in Figure 1, its cardinality and participation, and the foreign key that implements it in the relational schema (Section 4).

Relationship	Entities (1 side – N side)	Cardinality	Participation of N side	Foreign key in N-side table
contains	HOSTEL – ROOM	1 : N	Every room belongs to one hostel (total)	Hostel_ID
employs	HOSTEL – STAFF	1 : N	Every staff member is posted to one hostel (total)	Hostel_ID
receives	STUDENT – ROOM_ALLOCATION	1 : N	Every allocation belongs to one student (total)	Student_ID
assigned in	ROOM – ROOM_ALLOCATION	1 : N	Every allocation refers to one room (total)	Room_ID
allocates	STAFF – ROOM_ALLOCATION	1 : N	Every allocation is made by one staff member (total)	Allocated_By
raises	STUDENT – COMPLAINT	1 : N	Every complaint is raised by one student (total)	Student_ID
handles	STAFF – COMPLAINT	0..1 : N	A complaint may be unassigned until the warden assigns it (partial)	Assigned_To
applies for	STUDENT – LEAVE_REQUEST	1 : N	Every leave request belongs to one student (total)	Student_ID
approves	STAFF – LEAVE_REQUEST	0..1 : N	Empty while the request is pending (partial)	Approved_By
requests	STUDENT – VISITOR_PASS	1 : N	Every pass is requested by one student (total)	Student_ID
authorizes	STAFF – VISITOR_PASS	0..1 : N	Empty while the pass is pending (partial)	Approved_By

Design notes

- Over time a student may stay in several rooms and a room holds several students, so STUDENT and ROOM have a many-to-many relationship. It is resolved through the associative entity ROOM_ALLOCATION, which also stores the bed number, dates and allocation status.
- A student may have many allocation records, but at most one with Status = 'Active'. Together with the capacity check on ROOM, this rule will be enforced by a trigger during implementation.
- Admins, wardens and maintenance staff share the same attributes and differ only in their duties, so they are stored in a single STAFF entity with a Role attribute. STAFF is linked to complaints, allocations and approvals through role-named foreign keys (Assigned_To, Allocated_By, Approved_By).

4. Relational Schema

Each entity in Figure 1 is mapped to one relation. Attribute names are identical to those in the ER diagram. For every relation, the key of each attribute is marked as **PK** (Primary Key), **FK** (Foreign Key) or **UNIQUE**, followed by the planned constraints. Foreign-key references are listed below each relation.

4.1 STUDENT

Stores the details of every hostel resident.

```
STUDENT(
   Student_ID              PK,
   Registration_No         UNIQUE,    NOT NULL
   Name,                              NOT NULL
   Email                   UNIQUE,    NOT NULL
   Phone,                             NOT NULL
   Gender,                            NOT NULL, CHECK (Gender IN ('Male', 'Female'))
   Year,                              CHECK (Year BETWEEN 1 AND 4)
   Branch,                            NOT NULL
   CGPA                               CHECK (CGPA BETWEEN 0 AND 10)
)
```

Foreign keys: none (referenced by other relations).

4.2 HOSTEL

Stores each hostel block and the gender category it accommodates.

```
HOSTEL(
   Hostel_ID               PK,
   Hostel_Name             UNIQUE,    NOT NULL
   Gender_Type,                       NOT NULL, CHECK (Gender_Type IN ('Male',
                                      'Female'))
   Location,                          NOT NULL
   Total_Floors                       CHECK (Total_Floors > 0)
)
```

Foreign keys: none (referenced by other relations).

4.3 ROOM

Stores rooms of each hostel. Occupied_Count is updated automatically whenever an allocation becomes active or is vacated.

```
ROOM(
   Room_ID              PK,
   Room_Number,                    NOT NULL
   Floor_No,                       CHECK (Floor_No >= 0)
   Room_Type,                      CHECK (Room_Type IN ('Single', 'Double',
                                   'Triple', 'Quad'))
   Capacity,                       NOT NULL, CHECK (Capacity > 0)
   Occupied_Count,                 DEFAULT 0, CHECK (Occupied_Count BETWEEN 0 AND
                                   Capacity)
   Hostel_ID            FK         NOT NULL
)
```

Foreign keys:

```
Hostel_ID FK REFERENCES HOSTEL(Hostel_ID)
```

Additional constraints:

- UNIQUE (Hostel_ID, Room_Number) – a room number is not repeated within a hostel.

4.4 STAFF

Stores admins, wardens and maintenance staff. Role decides what each staff member can do in the system.

```
STAFF(
   Staff_ID             PK,
   Name,                           NOT NULL
   Role,                           NOT NULL, CHECK (Role IN ('Admin', 'Warden',
                                   'Maintenance'))
   Specialization,                 CHECK (Specialization IN ('Electrical',
                                   'Plumbing', 'Carpentry', 'Housekeeping',
                                   'Network')); NULL for Admin/Warden
   Phone                UNIQUE,    NOT NULL
   Email                UNIQUE,
   Hostel_ID            FK         NOT NULL
)
```

Foreign keys:

```
Hostel_ID FK REFERENCES HOSTEL(Hostel_ID)
```

4.5 ROOM_ALLOCATION

Associative relation recording which bed of which room is allotted to which student, and by whom.

```
ROOM_ALLOCATION(
   Allocation_ID          PK,
   Student_ID             FK,        NOT NULL
   Room_ID                FK,        NOT NULL
   Allocated_By           FK,        NOT NULL
   Bed_Number,                       NOT NULL, CHECK (Bed_Number > 0)
   Allocation_Date,                  NOT NULL, DEFAULT CURRENT_DATE
   Vacate_Date,                      CHECK (Vacate_Date >= Allocation_Date)
   Status                            DEFAULT 'Active', CHECK (Status IN ('Active',
                                     'Vacated', 'Cancelled'))
)
```

Foreign keys:

```
Student_ID FK REFERENCES STUDENT(Student_ID)
Room_ID FK REFERENCES ROOM(Room_ID)
Allocated_By FK REFERENCES STAFF(Staff_ID)
```

Additional constraints:

- At most one 'Active' allocation per student, and one 'Active' allocation per (Room_ID, Bed_Number).
- Bed_Number must not exceed the Capacity of the referenced room.

4.6 COMPLAINT

Stores maintenance complaints raised by students and their assignment to maintenance staff.

```
COMPLAINT(
   Complaint_ID           PK,
   Student_ID             FK,        NOT NULL
   Assigned_To            FK,        NULL until assigned
   Category,                         NOT NULL, CHECK (Category IN ('Electrical',
                                     'Plumbing', 'Carpentry', 'Housekeeping',
                                     'Network', 'Other'))
   Description,                      NOT NULL
   Priority,                         DEFAULT 'Medium', CHECK (Priority IN ('Low',
                                     'Medium', 'High'))
   Status,                           DEFAULT 'Open', CHECK (Status IN ('Open',
                                     'Assigned', 'In Progress', 'Resolved'))
   Created_Date,                     NOT NULL, DEFAULT CURRENT_TIMESTAMP
   Resolved_Date,                    CHECK (Resolved_Date >= Created_Date)
   Remarks
)
```

Foreign keys:

```
Student_ID FK REFERENCES STUDENT(Student_ID)
Assigned_To FK REFERENCES STAFF(Staff_ID)
```

4.7 LEAVE_REQUEST

Stores leave applications of students and the warden decision on them.

```
LEAVE_REQUEST(
   Leave_ID                PK,
   Student_ID              FK,        NOT NULL
   Approved_By             FK,        NULL while pending
   Start_Date,                        NOT NULL
   End_Date,                          NOT NULL, CHECK (End_Date >= Start_Date)
   Reason,                            NOT NULL
   Destination,
   Applied_Date,                      DEFAULT CURRENT_DATE
   Status                             DEFAULT 'Pending', CHECK (Status IN ('Pending',
                                      'Approved', 'Rejected'))
)
```

Foreign keys:

```
Student_ID FK REFERENCES STUDENT(Student_ID)
Approved_By FK REFERENCES STAFF(Staff_ID)
```

4.8 VISITOR_PASS

Stores visitor pass requests, their approval and the actual entry and exit of visitors.

```
VISITOR_PASS(
   Pass_ID                 PK,
   Student_ID              FK,        NOT NULL
   Approved_By             FK,        NULL while pending
   Visitor_Name,                      NOT NULL
   Visitor_Phone,                     NOT NULL
   Relation_To_Student,
   Purpose,
   Visit_Date,                        NOT NULL
   Entry_Time,
   Exit_Time,                         CHECK (Exit_Time > Entry_Time)
   QR_Code                 UNIQUE,
   Status                             DEFAULT 'Pending', CHECK (Status IN ('Pending',
                                      'Approved', 'Rejected', 'Completed'))
)
```

Foreign keys:

```
Student_ID FK REFERENCES STUDENT(Student_ID)
Approved_By FK REFERENCES STAFF(Staff_ID)
```

4.9 Normal Form of the Schema

Every relation has a single-attribute surrogate primary key and every non-key attribute depends only on that key, so the schema is in Third Normal Form (3NF). Student, room and staff details are stored only once and are referenced elsewhere through foreign keys. The only derived value is ROOM.Occupied_Count, which is kept deliberately so that vacancy can be checked quickly; it will be maintained by a trigger on ROOM_ALLOCATION rather than updated manually, so it cannot drift from the allocation records.